Reporte Práctico-Experimentales Nro. 001

1. Datos Generales

Asignatura	Sistemas distribuidos
Ciclo	6to
Unidad	Envío de mensajes distribuido
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	Uno
Título de la Práctica	Análisis Colaborativo de las Arquitecturas Principales en un Sistema Distribuido
Nombre del Docente	José O. Guamán Q.
Estudiantes	Kevin Joel Nole Cabrera Jasiel Isaac Quezada Luna Viviana Elizabeth Cordova Celi Isauro Michael Rivera Maldonado
Fecha	29 de septiembre de 2026 02 de octubre de 2026
Horario	Martes 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3

2. Objetivo(s) de la Práctica:

Analizar de manera colaborativa las arquitecturas fundamentales de los sistemas distribuidos (cliente-servidor, peer-to-peer, multicapa y microservicios), identificando sus características, ventajas, desventajas y casos de uso, mediante trabajo en equipo y el diseño de un caso de estudio

- Identificar las características estructurales y funcionales de las arquitecturas cliente-servidor, peer-to-peer, multicapa (n-tier), y microservicios en sistemas distribuidos.
- Comparar las ventajas, desventajas, escalabilidad y tolerancia a fallos de cada arquitectura mediante el llenado colaborativo de una matriz de análisis.
- Diseñar el diagrama de un caso de estudio propuesto aplicando la arquitectura distribuida más adecuad a, justificando técnicamente la elección del equipo

3. Materiales y reactivos:

Diapositivas, y enunciados del problema

- Hardware:
  - Computadora personal con conexión a internet.
- Software:
  - Procesador de texto.

4. Equipos y herramientas

Computadora personal con conexión a internet
Guía de práctica.
Plantilla de matriz comparativa

5. Procedimiento / Metodología

Metodología seguida:

Fase 1: Redacción del Prompt
Se redactó el prompt de manera colaborativa con todos los integrantes del grupo, procurando incluir instrucciones claras y precisas para obtener una respuesta adecuada por parte de la IA generativa.

Fase 2: Revisión
Se revisó detalladamente la respuesta proporcionada por la IA, verificando la información obtenida y comprobando que los datos fueran coherentes y no presentaran información inventada o incorrecta.

Fase 3: Completando la matriz
Una vez analizado y validada la información solicitada, se procedió a completar la matriz proporcionada en el APE. En ella se registraron aspectos relacionados sobre estructura, ventajas, desventajas, escalabilidad y ejemplos reales de cada arquitectura.

Fase 4: Creacion de l Diagrama en Draw.io
Con base en la información previamente analizada, se procedió a elaborar los diagramas correspondientes a cada arquitectura utilizando la herramienta Draw.io. Los diagramas permitieron representar de manera gráfica y organizada la estructura y los principales componentes de cada arquitectura.

Fase 5: Socialización y matriz comparativa
Finalmente, se realizó la socialización de los resultados obtenidos por los diferentes grupos. Posteriormente, se completó una matriz comparativa con la información presentada por los demás compañeros durante sus exposiciones, con el propósito de identificar y contrastar las principales características de las arquitecturas estudiadas.

6. Resultados esperados:

Al finalizar la práctica, el estudiante deberá haber obtenido y documentado lo siguiente:

- Documento de investigación de la arquitectura asignada (definición, componentes, ventajas/desventajas, escalabilidad y 2 ejemplos reales).

Criterio	Cliente-Servidor	Peer-to-Peer	Multicapa	Microservicios
Estructura	Se compone de tres bloques fundamentales, el	Red de dispositivos conectados en donde cada	Una arquitectura multicapa se organiza en	Su estructura se compone de: El frontend que es la parte que

	Cliente(frontend), Servidor(backend).	uno actúa como cliente y servidor al mismo tiempo. Se clasifican en Puros(Sistema totalmente descentralizados) e Híbridos(Algún tipo de centralización para la gestión de los demás dispositivos)	tres niveles fundamentales Capa de presentación, corresponde a la interfaz de usuario /presentación Capa media: Lógica de negocio / servidor de aplicaciones Capa de almacenamiento: Base de datos /servidor	los usuarios ven, luego está el gateway que se encarga de enrutar las solicitudes de los usuarios a los respectivos microservicios quienes procesarán dicha solicitud interactuando finalmente con la propia base de datos de dicho microservicio [6]
Ventajas	- Es desacoplado, permite la separación de la vista y la lógica de negocio. - Permite conectar múltiples clientes (navegadores web, aplicaciones móviles).	- Es descentralizado, o no requiere de un administrador central. - Tolerancia a fallas dado a distribución de recursos en varios nodos - Todos los nodos correspondientes pueden comunicarse entre sí.	- Los datos de uso frecuente pueden almacenarse en cachés ubicados en la capa media lo que permite que la información se obtenga de manera mas rapida, ademas de reducir la cantidad de solicitudes que deben enviarse a la base de datos - Cada una de las capas tiene una responsabilidades, por lo que los cambios o actualizaciones a la lógica interna de una capa, no afecta	Aplica la división de tareas en un sistema permitiendo que existan módulos con funciones en específico que cubran los requerimientos del sistema [4] Como cada servicio es completamente independiente de los demás, estos se pueden actualizar, testear, desplegar, etc. sin afectar a los demás módulos [4][6] Cada servicio cuenta con sus propios lenguajes facilitando que los equipos puedan asumir por completo la responsabilidad de un ciclo de vida de un módulo [5]

			directamente a las demas	
Desventajas	- Es sujeta a congestión cuando muchos clientes realizan peticiones de forma concurrente. - Requiere de la disponibilidad continua del servidor, si el servidor falla los clientes pierden el acceso.	- Alta complejidad dado a que cada dispositivo debe portar un software que le permita actuar como cliente y servidor.Adicionalemte que en algunas arquitecturas requiere la sincronización adecuada de los mismos. - Tráfico ante problemas de optimización puede haber problemas de saturación.	- Dado que múltiples clientes tienden a depender de componentes compartidos en el servidor, una falla de un solo componente en la capa media puede poner en riesgos las funciones de todo el conjunto de aplicaciones cliente	Debido a tener distintos módulos los puntos inseguros aumentan poniendo en riesgo una filtración de datos [4] Como cada módulo consume recursos que sumando al consumo de los demás módulos revela un consumo altamente significativo [6] El intercambio constante de información entre los módulos llega a crear una red de comunicación compleja que debe ser administrada para solucionar problemas como la latencia [6][4]
Escalabilidad	Es escalable, y se pueden incrementar distintas mejoras como Escalabilidad horizontal, Escalabilidad Vertical	Alta escalabilidad dado a que entre más usuarios se conecten más servidores y recursos existen. Teóricamente tiene una escalabilidad infinita.	Distribución de las carga entre múltiples capas, lo que facilita el crecimiento del sistema cuando aumenta el número de solicitudes	Este tipo de arquitectura tiene una alta escalabilidad ya que pueden invertirse recursos según se necesite en los servicios que ofrezca el sistema
Ejemplos reales	- Sistema Web del departamento	- Dentro del ámbito de Criptomonedas y	Sistema Universitario: gestión de matriculas	Dentro del ámbito bancario con las aplicaciones de

	- Médico de la UPS Campus[1] - Aplicación TaskMaster, es una plataforma de gestión colaborativa basada en el modelo cliente-servidor.[2]	Blockchain, en sectores tales como Bitcoin o Ethereum, las transacciones pasan por todos los nodos de la red para verificar su validez, ganando seguridad y descentralización sin depender de una entidad externa como un banco[3].	cumple con las tres capas principales, presentación, lógica de negocio y la capa de datos.	billeteras digitales como lo es "*deuna*" que es una aplicación basada en microservicios [7] Otro ejemplo de aplicación de esta arquitectura son las aplicaciones de entregas de comida a domicilio como es el caso de *"Delivery"* que se basa en este tipo de arquitectura [8]

- Un diagrama digital de la arquitectura asignada (draw.io o similar).

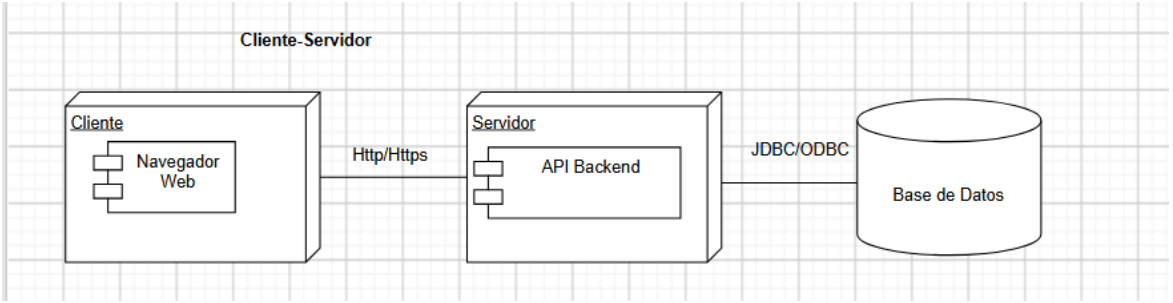


Fig. 1 Arquitectura Cliente-Servidor

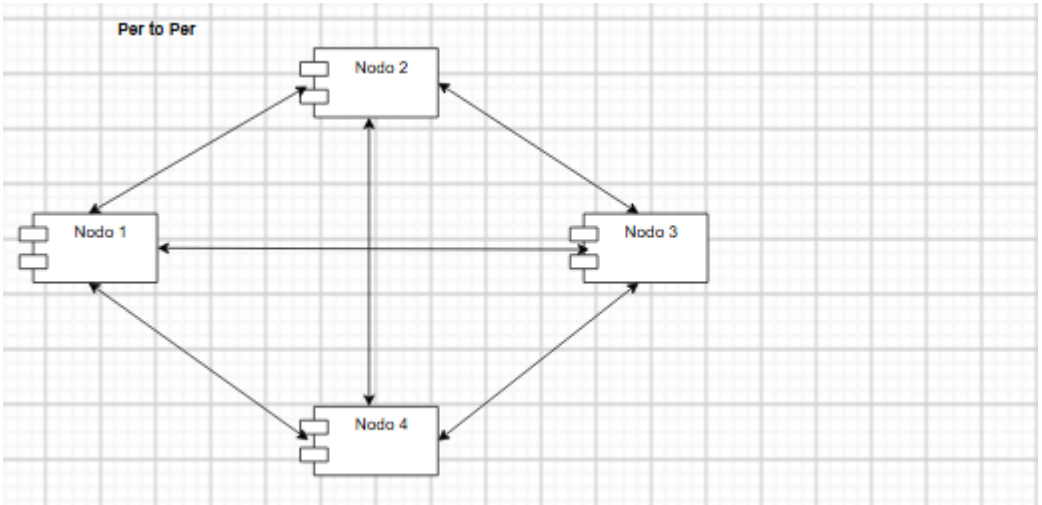


Fig. 2 Arquitectura Per to Per

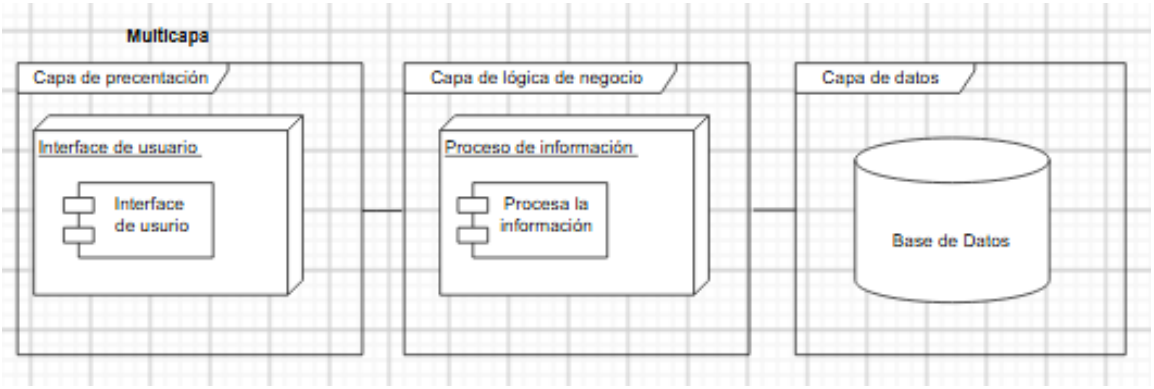


Fig. 3 Arquitectura Multicapas

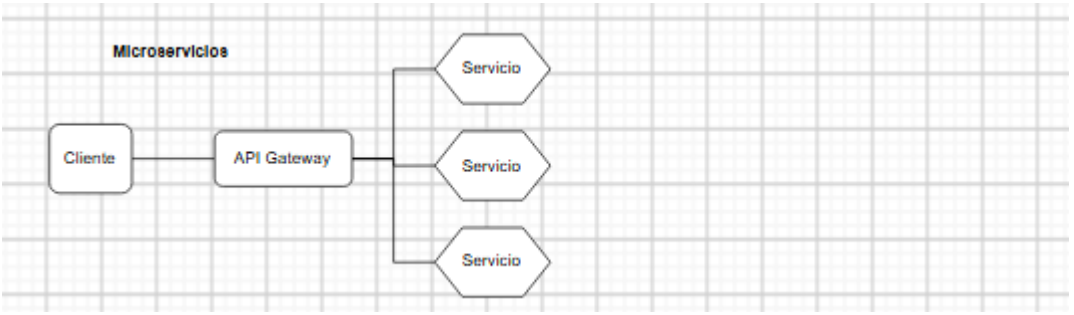


Fig. 4 Arquitectura Microservicio

- Matriz comparativa grupal completa (integrando las cuatro arquitecturas con los aportes de todos los equipos).

GRUPO	Criterio	Cliente-Servidor	Peer-to-Peer	Multicapa	Microservicios
	Estructura	Existen dos nodos cliente y servidor.	Todo los dispositivos actúan como clientes y servidores	Esta arquitectura, es una que se divide en tres capas principales, la capa de presentación, proceso y servicio	Los microservicios tienen divididos las funciones cumpliendo las reglas de negocio en conjunto siendo importante definir la comunicación y función

GRUPO 1 ...	Ventajas	Simplifica la administración	Tienen una escalabilidad ilimitada Cada nodo puede tener información	Se pueden distribuir las diferentes cargas dentro del sistema	Debido a su división dentro de las funciones se pueden dar mantenimiento por separado
	Desventajas	Muchas peticiones se satura el servidor	Alta volatilidad de nodos Latencia de transmisión de información	Las dependencia de los componentes	Puede llegar a causar alta latencia debido a la distribución que tienen Para la detección de errores hay que hacer todo un seguimiento
	Escalabilidad	Proporcionando mas servidores	Escalabilidad alta y	Por su tipo de arquitectura	Limitada por el presupuesto solo

		se obtienen mas respuestas	global	su escalabilidad es mucho mas practica y manejo de actualizaciones	se necesita tener más instancias de hardware
	Ejemplos reales	Sistema Académica, el proceso de matriculación del SGA.	Amazon y Netflix son ejemplos comunes.	Un sistema de gestión bancario, aplica una arquitectura multicapa	Aplicacion de soundcloud
GRUPO 2	Estructura	La estructura es desacoplada	Existen 4 modelos las distribuidas(cada nodo se conecta de forma aleatoria), no estructurada, híbrida y pura(tiene una estructura no centralizada)	Varias capa horizontales con un rol específico	Estructurado por servicios y por cada uno una base de datos
	Ventajas	Mejora la parte del desacoplamiento.	No depende de un servidor. Tolerancia a fallos.	La separación por su propia lógica y aislamiento Pruebas o simulaciones	Aplica el divide y vencerás pudiendo dividir de forma explicita y no mezclar responsabilidades
	Desventajas	Si ocurren muchas respuestas el servidor puede caerse.	Problemas de autenticación y detección de malware	Antipatrón cuando existen muchas peticiones, afecta rendimiento y latencia	El costo de mantener todos los módulos pueden a llegar a ser siginifcativos
	Escalabilidad	La escalabilidad es alta.	Tiene una escalabilidad moderada dado a que puede unirse varios nodos	Separación de responsabilidades	
	Ejemplos reales	Netflix, Youtube, el cual el navegador será el cliente	BitTorrent: en el que cada cliente actúa como almacenamiento de peliculas para que otros clientes los descarguen.	Gestión de salud, bibliotecas o librerías que se organizan en una arquitectura multicapas	
Grupo 3	Estructura	Maneja cliente y servidor	Esta cuenta con una arquitectura el cual cuenta con	Un sistema se distribuye en diferentes capas	La parte encargada de comunicar y encaminar es la API gateway

			nodos en el que todos actúan como clientes y servidores		
	Ventajas	Aislado el cliente del servidor	Alta Escalabilidad Tolerancia a fallos y caídas del sistema	Cada capa se maneja individualmente	Cada microservicio se puede actualizar o dar mantenimiento se puede hacer sin afectar al sistema completo
	Desventajas	Requiere que el servidor este siempre operativo	Nivel de complejidad Elevado Problemas de saturación de datos ante problemas de optimización.	Incorporar demasiadas capas un proceso sencillo generando latencia	El costo global de mantener todo el sistema puede llegar a ser bastante significativo
	Escalabilidad	Se puede agregar mas recursos de hardware	Alta dada la tendencia a ser infinito por la flexibilidad de dicha arquitectura	Por su desarrollo en capas es escalable	Tiene escalabilidad horizontal y vertical
	Ejemplos reales	Celular	Netflix	Banco	billeteras digitales
Grupo 5	Estructura	Los cliente realizan peticiones y el servidor responde.	Todos los nodos tienden actuar como cliente y servidor	Se dividen en capas con diferentes funcionalidades	Cada microservicio puede llegar a tener su propia base de datos o llegar a compartir
	Ventajas	El desacoplamiento	Cuando tienda a caer un nodo del sistema, esta arquitectura agarra los recursos de otro nodo	Son capas independiente	Cada módulo tiene su propio manejo y si falla uno de ellos el sistema no cae
	Desventajas	Al realizar muchas peticiones de los usuarios el servidor puede colapsar.	Una severa complejidad dado a la sincronización que requieren los dispositivos	El flujo es dependiente y puede traer problemas de latencia	La fuga de datos es mas probable si no hay seguridad en cada módulo
	Escalabilidad	Se puede agregar recursos al servidor.	Gran escalabilidad dado a que la inclusión de nuevos nodos tienden a	Es escalable con el tiempo	Alta limitada por el presupuesto de las empresas

			ser más recursos para el sistema		
	Ejemplos reales		BitTorret	Bascas moviles	Banco
Grupo 6	Estructura	Se basa en cliente y servidor, el cliente envía solicitudes y el servidor responde.		Un sistema dividido en tres capas principales	Hay una API gateway que encamina cada solicitud a los distintos microservicios
	Ventajas	Permite conectar múltiples clientes.		Mantenimiento aislado	Pueden dividir cada responsabilidad de forma clara y concisa
	Desventajas	La desventaja es que esos mismos cliente si hacen muchas peticiones el servidor puede colapsar.		Complejidad de implementación y alta dependencias	El costo de mantenimiento elevado
	Escalabilidad	Se puede agregar mas recursos de hardware al servidor.		Son escalables con el tiempo	depende de la finalidad del sistema pero pueden llegar a ser de escalabilidad horizontal y vertical
	Ejemplos reales	Gmail, el usuario hace una petición y Gmail te devuelve una respuesta		Sistemas bancarios. delivery	Sistemas de organizaciones empresariales (ERP)
Grupo 7	Estructura	Se basa en el cliente servidor.		Tres diferentes capas	Su estructura es detallada ya que presenta la parte de interfaz, el gateway y los microservicios
	Ventajas	Permite conectar muchos clientes.		Facilidad de elegir servicios o funciones por cada capa	Por cada servicio se puede modificar de forma independiente sin afectar al sistema
	Desventajas	Saturamiento, si se recibe demasiada solicitudes.		Trabaja como monolito, llevando a que se colapsan de todas	Manejar distintos microservicios puede llegar a ser complicado
	Escalabilidad			Escalabilidad media	se puede mejorar los microservicios segun sea la necesidad del momento
	Ejemplos reales	Banco de Loja, la app de la banca.		Amazon	Amazon

- Reporte individual breve (máx. 1 página) con conclusiones sobre qué arquitectura considera más versátil y por qué.

La arquitectura peer to peer consideramos que es bien versátil dado a que

el hecho de no tener una dependencia centralizada, debido a que transforma cada nodo de la red en un elemento que actúa como cliente y servidor, siendo así un recurso adicional para dicha arquitectura. Lo que permite tener una adaptabilidad adecuada de rendimiento, infraestructura y costos[3].

Adicionalmente que la carencia de un punto único de fallo garantiza la continuidad del servicio ante desconexiones masivas[3].

7. **Preguntas de Control:**

¿Qué es un sistema distribuido y cuáles son sus características fundamentales que lo diferencian de un sistema centralizado?

Un sistema distribuido se puede definir como un sistema en el cual toda su infraestructura no está en un mismo lugar sino que puede llegar a formar toda una red completa de nodos que se comunican entre sí con la característica de que al usuario final lo interprete como una sola entidad [7]

Las características más representativas que diferencian de un sistema distribuido de uno centralizado son:

Comunicación: como en un sistema distribuido sus procesos se encuentran dispersos la comunicación de los mismos debe ser a través de redes utilizando protocolos como por ejemplo: HTTP/REST, colas de mensaje como rabbitMQ [4] [7]

Gestión de datos: como cada módulo propuesto en el sistema es independiente de los demás la gestión de los datos se maneja de igual manera para así poder garantizar un bajo acoplamiento [5][6]

Variedad: en los sistemas distribuidos se puede llegar a usar distintas bases de código permitiendo que los módulos lleguen a usar diferentes lenguajes de programación [4][7]

Despliegue: el despliegue es independiente ya que los módulos son independientes pudiendo dar mantenimiento a microservicios sin necesidad de redesplegar el sistema [4][7]

Escalable: permite una escalabilidad granular permitiendo invertir recursos en los microservicios que lo requieran [4][7]

Explique la diferencia de comunicación entre la arquitectura cliente-servidor y la arquitectura peer-to-peer.

La diferencia radica en que la arquitectura peer-to-peer los clientes pueden ser servidores, mientras que en la arquitectura cliente-servidor las tarea se distribuye en que el cliente solo usa la interfaz y realiza peticiones y el servidor responde a esa petición.

¿Qué rol cumple el middleware en un sistema distribuido y en qué capa se ubica?

Se ubica en la capa de presentación, cumple un rol fundamental de intermediación, abstracción, seguridad y desacoplamiento entre las aplicaciones cliente y las capas internas del backend.

En la arquitectura de tres capas (3-tier), ¿qué función cumple cada capa y por qué separarlas mejora el mantenimiento?

Se organiza dividiendo en tres niveles, la capa de presentación que gestiona la interfaz gráfica, la capa de Negocio aquí se programa la lógica de negocio, validaciones, y algoritmos, y la capa de datos se encarga de gestionar la interacción con los servicios de base de datos. Al separarlas se separa las responsabilidades y es desacoplada, evita la desorganizacion del codigo[2].

¿Cuáles son las principales ventajas de los microservicios frente a un diseño monolítico, y qué desafíos introducen?

Las principales ventajas que ofrece la arquitectura de microservicios es que aplica la famosa frase “divide y vencerás” asignando a cada módulo una funcionalidad y también al tener dividido las responsabilidades se puede invertir los recursos de forma variable mientras que al entrar a diseños monolíticos estos son susceptibles a cualquier cambio ya que al tener todo en conjunto obligatoriamente se debe bajar el sistema para dar mantenimiento, corregir errores, etc. También en los diseños monolíticos en picos de solicitudes se pueden llegar a caer mientras que en arquitecturas de microservicios estos pueden simplemente a no proveer el servicio demandado y no todo el sistema [4][5]

¿Qué arquitectura consideraría más adecuada para una aplicación como BitTorrent y por qué?

La arquitectura adecuada y a la que actualmente corresponde es de tipo P2P(peer to peer), dado que es adecuado para distribuir archivos grandes de manera descentralizada y con una escalabilidad muy adecuada para el tipo de servicios y manejos de archivos que lleva[9].

¿Cómo influye la escalabilidad horizontal y vertical en la elección de una arquitectura distribuida?

Los diferentes tipos de escalabilidad horizontal o vertical cuando se trata de aplicaciones o sistemas que tienden a crecer a gran escala llegando a manejar datos masivamente es más conveniente usar una escalabilidad horizontal, permitiendo distribuir sus solicitudes mediante el balance de carga, la repartición de trabajos en varios nodos sin degradar el rendimiento

Por otro lado la escalabilidad vertical es más aplicada cuando se dispone de recursos físicos

¿Qué significa transparencia en un sistema distribuido y mencione dos tipos de transparencia?

La transparencia en un sistema distribuido se defino como la capacidad del sistema de esconder la separación y distribución de componentes ante el usuario o desarrollador, hace que múltiples computadoras se comporten como un único sistema centralizado.

Existen dos tipos:

- **Transparencia de Localización:** Permite el acceso de recursos del sistema a los usuarios ya sea archivos, bases de datos o algún otro tipo de servicios, sin necesidad de saber la ubicación exacta del nodo físico, dado a que esto se realiza mediante un identificador único abstracto.
- **Transparencia de Replicación:**Ocultan el hecho de que existen varias copias de un mismo recurso distribuidos por los distintos nodos pertenecientes al sistema, estos se sincronizan automáticamente para mejorar la disponibilidad y el rendimiento.

Si un servidor central falla, ¿qué arquitectura continúa funcionando y por qué?

En una red con arquitectura Peer to Peer todos los nodos o procesos desempeñan funciones equivalentes e interactuar cooperativamente como sin una distinción rígida(clientes y servidor), si un nodo llega a fallar, el resto de la red puede absorber la carga, por lo que se considera tolerante a fallos, convirtiendo en una arquitectura ideal cuando se presentan estos fallos

En su caso de estudio, ¿qué criterios fueron decisivos para elegir la arquitectura propuesta?

Multicapa, la arquitectura multicapa fue ideal y la considere una muy buena elección, esta arquitectura es adaptable a múltiples sistemas y servicios, con recursos y funcionalidades independientes, claro que pueden llegar a tener ciertas dependencias entre sí, lo que genera problemas en lo que respecta a las solicitudes de cliente, a pesar de eso, considero que es una ideal para la escalabilidad al paso del tiempo.

8. Bibliografía

[1] A. A. Llano Jiménez, "Análisis, diseño, desarrollo e implementación de un sistema Web de control de historias y citas médicas para el Departamento médico de la UPS Campus Sur," Tesis de grado, Universidad Politécnica Salesiana, Quito, Ecuador, 2018.

[2] D. A. Rosales Flores, "Análisis comparativo de frameworks frontend: Eficiencia y facilidad de uso entre React y Svelte en el desarrollo de aplicaciones web," Examen complexivo, Universidad Estatal Península de Santa Elena, La Libertad, Ecuador, 2025.

[3] Nakamoto, S. (2019). Bitcoin: A Peer-to-Peer Electronic Cash System. 1(1), 9. https://bitcoin.org/bitcoin.pdf

[4] D. López y E. Maya, "Arquitectura de Software basada en Microservicios para Desarrollo de Aplicaciones Web," en *Séptima Conferencia de Directores de Tecnología de Información (TICAL 2017)*, San José, Costa Rica, jul. 2017

[5] D. A. Barrios Contreras, "Arquitectura de microservicios," *TIA (Tecnología, Investigación y Academia)*, vol. 6, no. 1, pp. 36–46, ene.-jun. 2018

[6] Á. I. Rodríguez, J. I. Padilla y H. A. Parra, "Arquitectura basada en micro-servicios para aplicaciones web," *TIA (Tecnología, Investigación y Academia)*, vol. 7, no. 2, pp. 12–20, jul.-dic. 2019

[7] Deuna: Una arquitectura basada en microservicios". IT ahora. Accedido el 29 de septiembre de 2026. [En línea]. Disponible: https://itahora.com/2024/10/03/deuna-una-arquitectura-basada-en-microservicios/

[8] “Food Delivery App Architecture Explained For Foodtech Leaders”. Folio3 FoodTech. Accedido el 29 de septiembre de 2026. [En línea]. Disponible: https://foodtech.folio3.com/blog/food-delivery-app-architecture-explained/

[9]Pouwelese, J., & Garbacki, P. (2020, 15 de enero). *THE BITTORRENT P2P FILE-SHARING SYSTEM: MEASUREMENTS AND ANALYSIS*. https://www.researchgate.net/publication/221160480_The_Bittorrent_P2P_File-Sharing_System_Measurements_and_Analysis

[10]Scalahed, “Sistemas distribuidos,” Plataforma SCALA, n.d. [En línea]. Disponible en: https://gc.scalahed.com/recursos/files/r161r/w25474w/SistemasDistribuidos.pdf [Accedido: 02-oct-2026].

9. Declaración del uso de IA

Declaró que, para la elaboración de esta tarea, hice uso de una herramienta de Inteligencia Artificial (Notebook) únicamente como apoyo para mejorar la redacción y síntesis de ideas.

El uso de la IA se realizó respetando el Contrato de uso ético y responsable de la Inteligencia Artificial, comprometiéndose a:

- No copiar directamente sin comprensión de los textos generados.
- Revisar, adaptar y redactar las ideas en nuestras propias palabras.
- Mantener un aprendizaje activo y crítico en el desarrollo del trabajo.

- **Prompt usado en Microservicios**

La IA que se usó en el desarrollo de esta práctica fue Notebook con el siguiente prompt

CONTEXTO (C)
Analizar y estructurar la base conceptual teórica de la arquitectura de microservicios y sistemas distribuidos basándote únicamente en las fuentes de este cuaderno de NotebookLM.

ROL (R)
Arquitecto Principal de Software Científico y Doctor en Ciencias de la Computación, experto en teoría de sistemas distribuidos, patrones de arquitectura, consistencia de datos y resiliencia.

ACCIÓN (A)
Basándote en las fuentes provistas, realiza los siguientes pasos secuenciales:
1. Define conceptos y teoremas clave (CAP, PACELC, Relojes Lógicos, Modelos de Consistencia, Consensus Protocols).
2. Analiza propiedades de diseño: Bounded Contexts, Acoplamiento/Cohesión, Autonomía y Transacciones Distribuidas (Sagas vs 2PC).
3. Explica modelos teóricos de fallos (Partition Tolerance, Cascading Failures) y estrategias de resiliencia (Circuit Breaker, Bulkhead, Backpressure).
4. Analiza los trade-offs teóricos al migrar de monolito a microservicios.
5. Cita explícitamente los textos o autores del cuaderno que respaldan cada punto y señala brechas si falta información teórica.

FORMATO (F)
Documento Markdown estructurado en:

- 1. Resumen Ejecutivo Teórico
- 2. Marco Teórico y Teoremas
- 3. Descomposición y Dominio Arquitectónico
- 4. Tabla de Trade-offs (| Propiedad/Teorema | Monolito | Microservicios | Impacto |)
- 5. Citas y Hallazgos Clave

PÚBLICO OBJETIVO
Arquitectos de Software Senior e Investigadores que requieren justificación teórica rigurosa. Usa lenguaje académico y formal, evitando detalles de implementación de código.

- **Prompt usado en Peer to Peer**

La IA que se usó en el desarrollo de esta práctica fue NotebookLM con el siguiente prompt:

- **CONTEXTO**
  - Realizar un análisis y síntesis respecto a la arquitectura Peer-to-Peer (P2P) en el contexto de sistemas distribuidos en base a la documentación cargada.
- **ROL**
  - Estudiante investigador en Ciencias de la Computación, redes distribuidas, algoritmos y análisis de sistemas.
- **ACCIÓN**
  - Basándose de forma estricta en las fuentes provistas, realiza un análisis detallado abordando las siguientes áreas de estudio:
    - Definición general del concepto de arquitectura peer to peer, basada en la documentación.
    - Estructura, implementación y métricas para su aplicación en problemáticas reales.
    - Propósitos iniciales,clasificación , ventajas y desventajas

  - **PÚBLICO OBJETIVO**

    - Docentes y revisores del área de Arquitectura de Software de la carrera de Ingeniería en Ciencias de la Computación. El informe debe presentar un tono académico, técnico y formal, apto para la sustentación de una práctica universitaria.

- **Promt usado para consultar Cliente-Servidor**

Actúa como un investigador experto en Ingeniería de Software. Basándote únicamente en los documentos que te he proporcionado, realiza un resumen estructurado sobre el tema 'Arquitectura Cliente-Servidor', Tu respuesta debe estar dividida en las siguientes secciones: -

- Definición Técnica (¿Qué es?): Define el concepto basándote estrictamente en los autores de los
- ¿Para qué sirve?: Explica su propósito, los beneficios que aporta y qué problemas resuelve.

- ¿Cómo se implementa?: Describe las metodologías, métricas o pasos necesarios para aplicarla en proyectos reales según los textos.
- Ventajas
- Desventajas
- Es escalable?
- REGLA ESTRICTA DE CITACIÓN: Al final de cada párrafo o afirmación clave, debes incluir la referencia exacta de dónde extrajiste la información. Utiliza el formato: [Nombre del documento, Página/Sección]. Si alguna de estas seccioness no tiene respuesta en los documentos proporcionados, no inventes la información; simplemente indica: 'Los documentos proporcionados no contienen información sobre este punto'."

- **Prompt Usado - Cliente multicapas**
  La IA usada para la consulta de información fue NotebookLM, y se le cargo el siguiente prompt
  Basándose en los documentos que te cargue quiero que actúes como docente universitario que imparte la asignatura de Sistemas Distribuidos. Para la primera semana de clases estás explicando arquitecturas de software multicapas. quiero que me expliques todos los conceptos principales de esta arquitectura, considerando la siguiente estructuras: Definición, Estructura, Ventajas, Desventajas, Escalabilidad, un ejemplo práctico
  Utiliza un lenguaje académico, pero fácil de comprender. Evita el uso de términos técnicos, diferencia entre los conceptos, no inventes información, solo basate en la información que te proporciones

10. Conclusiones

- Mediante la elaboración de la matriz comparativa se pudieron identificar y contrastar las ventajas, desventajas, escalabilidad y tolerancia a fallos de las arquitecturas estudiadas. Este análisis permitió comprender que aspectos como la distribución de recursos, el nivel de dependencia entre componentes, la posibilidad de agregar nuevos nodos y la gestión de fallos son factores importantes al momento de seleccionar una arquitectura para un sistema distribuido.
- El diseño de los diagramas en Draw.io facilitó la comprensión de conceptos teóricos, especialmente de la separación de responsabilidades en las arquitecturas multicapa y microservicios, así como de la relación entre clientes y servidores en las arquitecturas cliente-servidor y peer-to-peer.